

REPORTE TAREA 1

ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Rodrigo Ariel Cáceres Gaete

7 de septiembre de 2026

23:46

1. Introducción

Este informe presenta un análisis experimental enfocado en dos problemas fundamentales de las ciencias de la computación: el ordenamiento de arreglos unidimensionales y la multiplicación de matrices cuadradas. El objetivo principal de este estudio es relacionar el comportamiento práctico de diversos algoritmos con su respectiva complejidad teórica asintótica. Mediante la evaluación rigurosa de los tiempos de ejecución y la huella de memoria bajo distintos escenarios, tamaños y topologías de datos, se busca evidenciar cómo factores del mundo real —tales como las constantes operativas ocultas y la gestión de recursos del sistema— impactan el rendimiento final de las implementaciones teóricamente óptimas.

2. Entorno experimental

Para garantizar la reproducibilidad y el correcto análisis empírico de los tiempos de ejecución y la gestión de memoria, todos los experimentos fueron realizados en un entorno de ejecución unificado. Las especificaciones detalladas del hardware y software utilizado se listan a continuación:

- **Procesador:** AMD Ryzen 5 2600 Six-Core Processor, 3.40 GHz (6 núcleos físicos, 12 procesadores lógicos).
- **Memoria RAM:** 16.0 GB.
- **Sistema Operativo:** Microsoft Windows 10 Education.

- **Compilador:** g++ (GCC) versión 15.2.0.

Los conjuntos de datos (datasets) generados para la evaluación se ciñen a las especificaciones del enunciado de la tarea y se dividen según el problema a resolver. Para el análisis de algoritmos de ordenamiento, se instanciaron arreglos unidimensionales con tamaños que varían logarítmicamente desde $n = 10$ hasta $n = 10^7$ elementos. Estos arreglos fueron generados bajo tres distribuciones iniciales para estresar los algoritmos: elementos en orden aleatorio, en orden estrictamente ascendente y en orden estrictamente descendente.

Por otro lado, para la evaluación de los algoritmos de multiplicación, se construyeron matrices cuadradas de dimensión $n \times n$, con n variando entre 16 y 256. Con el fin de observar cómo la disposición espacial afecta el rendimiento, estas matrices se estructuraron en tres tipologías: densas (con valores distribuidos en la totalidad de la estructura), diagonales (con valores no nulos restringidos exclusivamente a la diagonal principal) y dispersas (con una alta proporción de ceros en el dominio matricial).

3. Resultados y Análisis

En esta sección se presentan y discuten los resultados obtenidos en las mediciones empíricas. Para el análisis de complejidad, se han seleccionado gráficas en escala logarítmica para ambos ejes, lo que permite visualizar las diferencias en las tasas de crecimiento de los distintos algoritmos evaluados.

3.1. Algoritmos de Ordenamiento

Para analizar el rendimiento práctico del ordenamiento, la Tabla 1 resume los tiempos promedio de ejecución para la instancia más grande evaluada ($n = 10^7$).

Cuadro 1: Tiempo de ejecución promedio (ms) para $n = 10^7$.

Algoritmo	Aleatorio	Ascendente	Descendente
Merge Sort	11825.20	10048.85	9980.79
Patience Sort	8648.03	11235.24	2505.78
<code>std::sort</code>	2051.06	1244.50	1151.65

Como se observa en la Tabla 1, `std::sort` supera holgadamente a las implementaciones manuales, siendo entre 4 y 5 veces más rápido. Esto demuestra el profundo nivel de optimización de la Standard Template Library (STL) de C++. Además, es notable la sensibilidad de Patience Sort a la distribución inicial: es significativamente más veloz en arreglos descendentes comparado con los ascendentes.

Para evaluar el crecimiento asintótico, la Figura 1 presenta la evolución temporal del ordenamiento bajo escenarios de entrada aleatoria y ascendente.

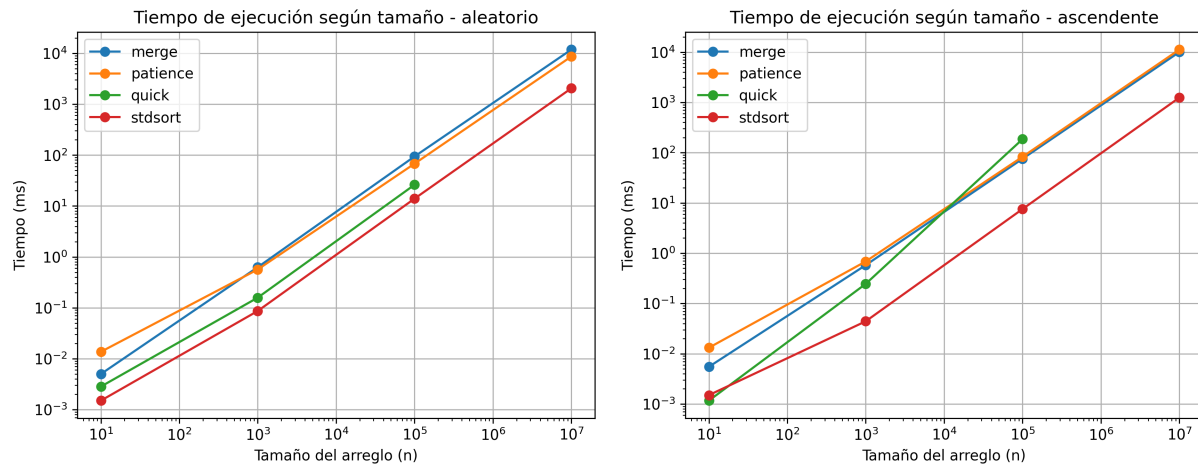


Figura 1: Tiempos de ejecución según tamaño de entrada (escala log-log). Izquierda: Arreglos aleatorios. Derecha: Arreglos ascendentes.

En la Figura 1, se confirma que todos los algoritmos presentan una tendencia lineal-logarítmica en escala log-log, consecuente con la complejidad teórica $\mathcal{O}(n \log n)$ esperada. Las líneas mantienen pendientes similares; la diferencia principal radica en su desplazamiento vertical, el cual está determinado por las constantes ocultas en las operaciones base de cada método y por la eficiencia en el uso de la memoria caché.

3.2. Multiplicación de Matrices

Para la multiplicación de matrices, la Tabla 2 muestra los tiempos obtenidos en el dominio máximo evaluado ($n = 256$).

Cuadro 2: Tiempo de ejecución promedio (ms) para $n = 256$.

Algoritmo	Densa	Diagonal	Dispersa
Naive	472.93	470.43	486.41
Strassen	15679.18	15765.45	15860.05

A diferencia del caso de ordenamiento, la topología de la matriz (densa, diagonal o dispersa) no generó diferencias significativas de tiempo intragrupo. Sin embargo, el contraste entre algoritmos es drástico: el enfoque clásico (Naive) es más de 30 veces más rápido que Strassen para $n = 256$.

Este fenómeno se visualiza claramente en la Figura 2, donde graficamos la curva de crecimiento temporal de ambos métodos.

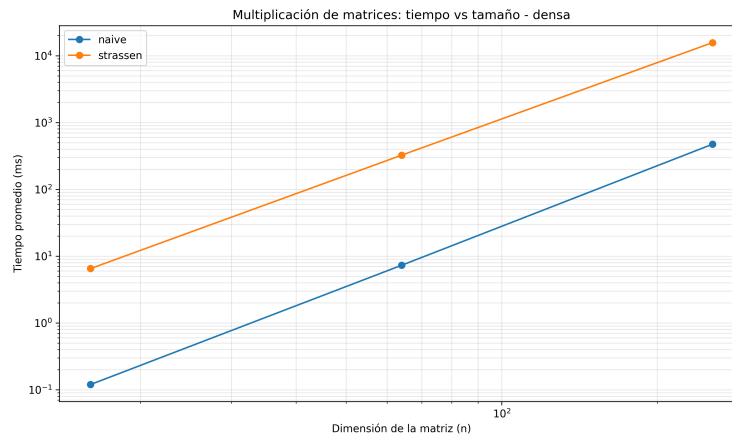


Figura 2: Tiempo de ejecución versus dimensión de la matriz (n) para matrices densas.

Aunque el algoritmo de Strassen reduce la complejidad asintótica del $\mathcal{O}(n^3)$ estándar a $\mathcal{O}(n^{\log_2 7}) \approx \mathcal{O}(n^{2.81})$, la Figura 2 evidencia que las constantes ocultas asociadas a la reserva de memoria, copias de sub-arreglos y las llamadas recursivas son masivas. Hasta el dominio evaluado de $n = 256$, la ventaja del exponente asintótico no es capaz de compensar el peso operativo, demostrando que Strassen solo sería práctico cruzando un umbral de n considerablemente mayor.

4. Conclusiones

La extensión máxima para esta sección es un párrafo breve (máximo 1/4 de página).

Los resultados experimentales demuestran que la superioridad asintótica teórica no siempre dicta el rendimiento práctico en instancias de tamaño moderado. En el problema de ordenamiento, la implementación de biblioteca `std::sort` probó ser la más robusta y veloz en todas las distribuciones de datos evaluadas, minimizando además el consumo de memoria; contrastando fuertemente con algoritmos como *Patience Sort*, cuyo tiempo se disparó en arreglos aleatorios. Por su parte, la multiplicación de matrices evidenció el peso de las constantes ocultas: a pesar de que el algoritmo de Strassen posee una mejor complejidad algorítmica teórica ($\mathcal{O}(n^{2.81})$ frente al $\mathcal{O}(n^3)$ del método clásico), la sobrecarga computacional de las llamadas recursivas y su mayor huella de memoria provocaron que la aproximación iterativa Naive fuera invariablemente más rápida para todos los tipos de matrices dentro del dominio estudiado (hasta $n = 256$). Esto confirma empíricamente que Strassen requiere de un *crossover point* considerablemente mayor para justificar su utilidad en la práctica.